

объектно-ориентированное программирование

принципы **SOLID** и **GRASP**

ЧТО ЭТО ТАКОЕ?

принципы SOLID и GRASP

- основные концепции ООП ← какие подходы есть чтобы писать ОО-код
- SOLID и GRASP ← правила, которые нужно соблюдать, чтобы ваш ОО-код был поддерживаемым и расширяемым
- SOLID – таблица умножения от мира ООП

МОДУЛИ

принципы SOLID и GRASP

- когда мы говорим “модуль”, мы имеем в виду...
 - какой-то тип в коде
 - инкапсулирующий логику и состояние
 - самодостаточная единица системы, реализующая определённый функционал

SOLID

single responsibility principle

**проектирование типов, таким образом что они
имеют единственную причину для изменения**



single responsibility principle

правила для...

SRP

- корректной инкапсуляции логики

пример нарушения SRP

```
public record OperationResult(...);

public class ReportGenerator
{
    public void GenerateExcelReport(OperationResult result)
    {
        ...
    }

    public void GeneratePdfReport(OperationResult result)
    {
        ...
    }
}
```

причины нарушения SRP

- “простота” (топорность)
нет необходимости в абстракциях
низкий порог для онбординга
- переиспользование логики
часто логика в типах не соблюдающих SRP имеет общие части, вызвать
приватный метод типа в нескольких местах проще чем реализовывать
грамотную декомпозицию

проблемы от нарушения SRP

- сильная связанность реализации различных бизнес требований от простого: загрязнённый контекст для IntelliSense до тяжёлого: усложнение тестирования
- усложнённая кастомизация отдельных реализаций изменения в общем коде могут поломать другие решения

как определить нарушение SRP

- наличие не связанной логики в одном типе
- наличие нескольких разных способов делать одно и то же в одном типе
- сильное различие в наборе данных для выполнения операции
- сильное различие в наборе зависимостей для выполнения операции
- по сути, один тип берёт на себя ответственность нескольких типов

как исправить нарушение SRP

- декомпозиция
- полиморфизм
- выделение новых абстракций

пример соблюдения SRP

```
public record OperationResult(...);

public interface IReportGenerator
{
    void GenerateReport(OperationResult result);
}

public class ExcelReportGenerator : IReportGenerator
{
    public void GenerateReport(OperationResult result)
    {
        ...
    }
}

public class PdfReportGenerator : IReportGenerator
{
    public void GenerateReport(OperationResult result)
    {
        ...
    }
}
```

преимущества от соблюдения SRP

- более чёткая объектная модель
- изоляция различных частей логики
- меньше вероятность оказать влияния на другие реализации при внесении изменений
- упрощение модульного тестирования

SOLID

open/closed principle

проектирование типов, таким образом что их логику можно расширять, не изменяя их исходный код
тип должен быть открытым для расширения, но закрытым для изменений



open/closed principle

правила для...

ОСР

- корректной инкапсуляции логики
- корректного использования полиморфизма

пример нарушения ОСР

```
public enum BinaryOperation
{
    Summation,
    Subtraction,
}

public class BinaryOperand
{
    private readonly int _left;
    private readonly int _right;

    // ...

    public int Evaluate(BinaryOperation operation)
    {
        return operation switch
        {
            BinaryOperation.Summation => _left + _right,
            BinaryOperation.Subtraction => _left - _right,
        };
    }
}
```

причины нарушения

ОСР

- некорректное разделение логики/выделение абстракций
- итеративная разработка без учета общей картины
 - функционал расширялся небольшими шагами
 - каждый шаг добавлял небольшую вариативность логики

проблемы от нарушения ОСР

- бОльшая связанность логики
- бОльшая вероятность повлиять на другие реализации
- бОльшая сложность модульного тестирования
 - нет возможности протестировать конкретную реализацию изолированно

как определить нарушение OCP

- SRP – про типы, OCP – про реализации
 - “нарушение SRP” на уровне реализации (метода) – нарушение OCP
- представьте что разрабатываете библиотеку
 - если потребители могут её расширить без изменения исходников – OCP соблюдается
 - если нет – не соблюдается

как исправить нарушение ОСР

- выделение абстракций
 - определите область вариативности в коде
 - выделите для неё интерфейс
- полиморфизм
- декомпозиция

пример соблюдения ОСР

```
public interface IBinaryOperation
{
    int Evaluate(int left, int right);
}

public class Summation : IBinaryOperation
{
    public int Evaluate(int left, int right)
        ⇒ left + right;
}

public class Subtraction : IBinaryOperation
{
    public int Evaluate(int left, int right)
        ⇒ left - right;
}
```

```
public sealed class BinaryOperand
{
    private readonly int _left;
    private readonly int _right;

    // ...

    public int Evaluate(IBinaryOperation operation)
        ⇒ operation.Evaluate(_left, _right);
}
```


преимущества от соблюдения ОСР

- изоляция реализаций
- возможность расширить логику без влияния на другие реализации
- упрощение модульного тестирования

SOLID

liskov substitution principle

проектирование иерархий типов, таким образом
что логика дочерних типов не нарушает инвариант
и интерфейс родительских типов



liskov substitution principle

правила для...

ОСР

- корректного использования наследования
- проектирования иерархий

пример нарушения LSP

```
public record Coordinate(int X, int Y);

public class Creature
{
    public void Die()
    {
        Console.WriteLine("I am dead now");
    }
}

public class Bird : Creature
{
    public virtual void FlyTo(Coordinate coordinate)
    {
        coordinate
        Console.WriteLine("I am flying");
    }
}

public class Penguin : Bird
{
    public override void FlyTo(Coordinate coordinate)
    {
        Die();
    }
}
```


пример нарушения LSP

```
StartMigration(birds, new Coordinate(123, 45));
```

```
void StartMigration(IEnumerable<Bird> birds, Coordinate coordinate)
{
    foreach (var bird in birds)
    {
        bird.FlyTo(coordinate);
    }
}
```

```
var birds = new[] { new Penguin() };
```

пример нарушения LSP

```
public class Bat : Creature
{
    public void FlyTo(Coordinate coordinate)
    {
        Console.WriteLine("I bat and am flying");
    }
}
```

```
void StartMigration(
    IEnumerable<Creature> creatures,
    Coordinate coordinate)
{
    foreach (var creature in creatures)
    {
        if (creature is Bird bird)
        {
            bird.FlyTo(coordinate);
        }

        if (creature is Bat bat)
        {
            bat.FlyTo(coordinate);
        }
    }
}
```

причины нарушения

LSP

- плохое понимание правил использования наследования
- попытка отражения фактических иерархий в объектной модели
 - объектная модель должна отражать предметную область
 - объектная модель должна быть приближена к предметной области
 - реализм $\neq$ хороший код
 - подходы проектирования $>$ попадание 1в1 в предметную область

проблемы от нарушения LSP

- сильная связанность между разными уровнями иерархий
 - (наличие уровней иерархий в принципе)
- возникновение special-case в бизнес логике
 - общая абстракция может быть корректна с точки зрения предметной области, но не корректна с точки зрения существующей бизнес логики
- нарушение OCP

```
if (creature is Bird bird)
{
    bird.FlyTo(coordinate);
}

if (creature is Bat bat)
{
    bat.FlyTo(coordinate);
}
```

как определить нарушение LSP

- реализация не соответствует ожиданиям поведения
 - неочевидная логика
 - исключения там, где их быть не должно
- костыли над полиморфизмом
 - использования интроспекции (`obj is SomeType`) для вариативности логики
- неявные связи между типами в иерархии
- “общие” поведения, не присущие всем дочерним типам

как исправить нарушение LSP

- ре-моделирование иерархии
 - пересмотр общих/частных типов
- использование feature-интерфейсов
 - интерфейсы, отражающие какую-то возможность/действие типа
 - использование их в качестве общих типов

пример соблюдения LSP

```
public record Coordinate(int X, int Y);

public interface ICreature
{
    void Die();
}

public interface IFlyingCreature : ICreature
{
    void FlyTo(Coordinate coordinate);
}

public class CreatureBase : ICreature
{
    public void Die()
    {
        Console.WriteLine("I am dead now");
    }
}
```

```
public class Bird : CreatureBase { }

public class Penguin : Bird { }

public class Colibri : Bird, IFlyingCreature
{
    public void FlyTo(Coordinate coordinate)
    {
        Console.WriteLine("I am colibri and I'm flying");
    }
}

public class Bat : CreatureBase, IFlyingCreature
{
    public void FlyTo(Coordinate coordinate)
    {
        Console.WriteLine("I am bat and I'm flying");
    }
}
```

пример соблюдения LSP

```
void StartMigration(IEnumerable<IFlyingCreature> creatures, Coordinate coordinate)
{
    foreach (var creature in creatures)
    {
        if (Random.Shared.NextDouble() < 0.8)
        {
            creature.FlyTo(coordinate);
        }
        else
        {
            creature.Die();
        }
    }
}

var creatures = new IFlyingCreature[] { new Colibri(), new Bat() };

StartMigration(creatures, new Coordinate(123, 45));
```


преимущества от соблюдения LSP

- явность реализации
- более низкая связанность между базовыми и конкретными типами

SOLID

interface segregation principle

проектирование маленьких абстракций, которые
ответственны за свой конкретный функционал, а не
одной всеобъемлющей, содержащий много различного



interface segregation principle

правила для...

ISP

- корректного проектирования интерфейсов

пример нарушения ISP

```
public interface ICanAllDevice
{
    void Print();

    void PlayMusic();

    void BakeBread();
}
```


проблемы от нарушения ISP

- абстракции обрастают лишними, не нужными для всех её пользователей, поведением
- это приводит к большему пространству для ошибок
- нарушение SRP у реализаций

как определить нарушение ISP

- ISP – SRP для интерфейсов, признаки такие же как для нарушения SRP

как исправить нарушение ISP

- декомпозиция

пример соблюдения ISP

```
public interface IPrinter
{
    void Print();
}
```

```
public interface IMusicPlayer
{
    void Play();
}
```

```
public interface IBakery
{
    void BakeBread();
}
```

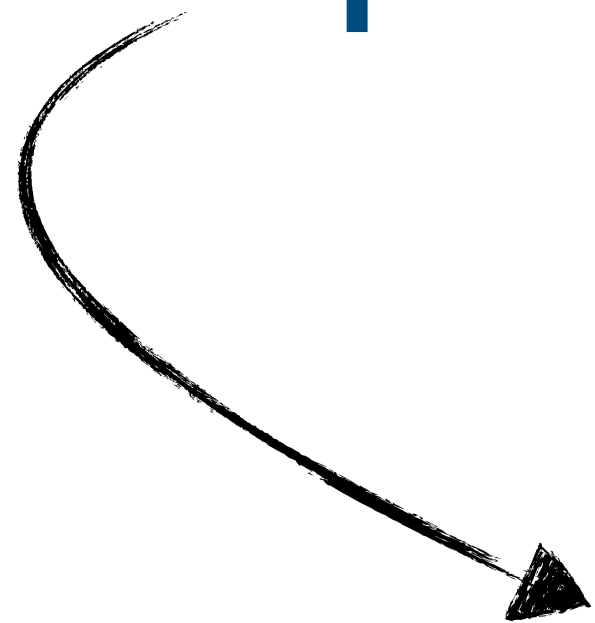
преимущества от соблюдения ISP

- соблюдение SRP у реализаций
- отсутствие “лишнего” функционала при работе с объектами абстракций
 - меньше протечки деталей реализации
- БОльшая модульность
 - после декомпозиции интерфейсов их может реализовывать один тип
 - а может несколько
 - а может одну часть один, а другую часть другой

SOLID

dependency inversion principle

проектирование типов, таким образом что одни реализации не зависят от других напрямую



dependency inversion principle

правила для...

ISP

- корректного проектирования связей между типами

виды абстракций

DIP

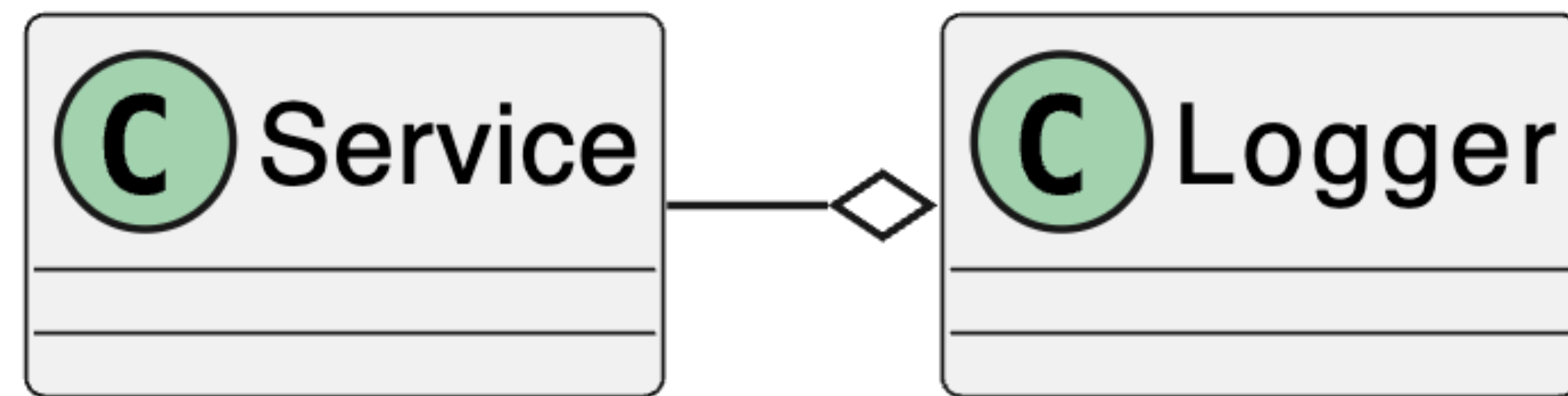
- верхнеуровневая/низкоуровневая
 - разница в логическом отношении между функционалом
 - низкоуровневая – бизнес правила, которые могут быть переиспользованы
 - верхнеуровневая – конкретный сценарий, оркестрация низкоуровневых абстракций

ВИДЫ абстракций

DIP

- бизнесовая/инфраструктурная
 - разница в назначении абстракции
 - бизнесовая – отражает какую-то бизнес-логику, описывает предметную область
 - инфраструктурная – код, не являющийся частью предметной области, но необходимый для работы приложения

пример нарушения DIP



пример нарушения DIP

```
public interface IReportStorage
{
    UploadReportResult UploadReport(Report report);
}

public class FileSystemReportStorage : IReportStorage;

public class NetworkReportStorage : IReportStorage;
```

пример нарушения DIP

```
public abstract record UploadReportResult
{
    private UploadReportResult() { }

    public sealed record Success : UploadReportResult;

    public sealed record DirectoryNotFound : UploadReportResult;

    public sealed record ServerNotFound : UploadReportResult;

    public sealed record NetworkError : UploadReportResult;
}
```

чуть глубже чем кажется

DIP

- нарушение этого принципа не обязательно происходит при явных связях
 - излишняя связь между реализациями может отражаться не на уровне связей между типами, а на уровне того как сделана реализация
 - методы одного типа могут быть реализованы в соответствии с логикой другого типа

проблемы от нарушения DIP

- сильная связанность между абстракциями различного уровня/назначения
- детали реализации одной абстракции начинают влиять на другую
 - больше вероятность что-то сломать
- усложнение модульного тестирования
 - отсутствие возможности использовать mock-объекты

как определить нарушение DIP

- явные связи между реализациями разного уровня/назначения
 - важно: связи, которых не должно быть
 - данный принцип не означает, что везде должны быть интерфейсы
- неявные связи между различными реализациями
 - если код в одном месте написан с расчётом на определённую реализацию в другом
- наличие связи между абстракцией и её реализацией

как исправить нарушение DIP

- выделение абстракций / ре-проектирование абстракций
- декомпозиция

пример соблюдения DIP

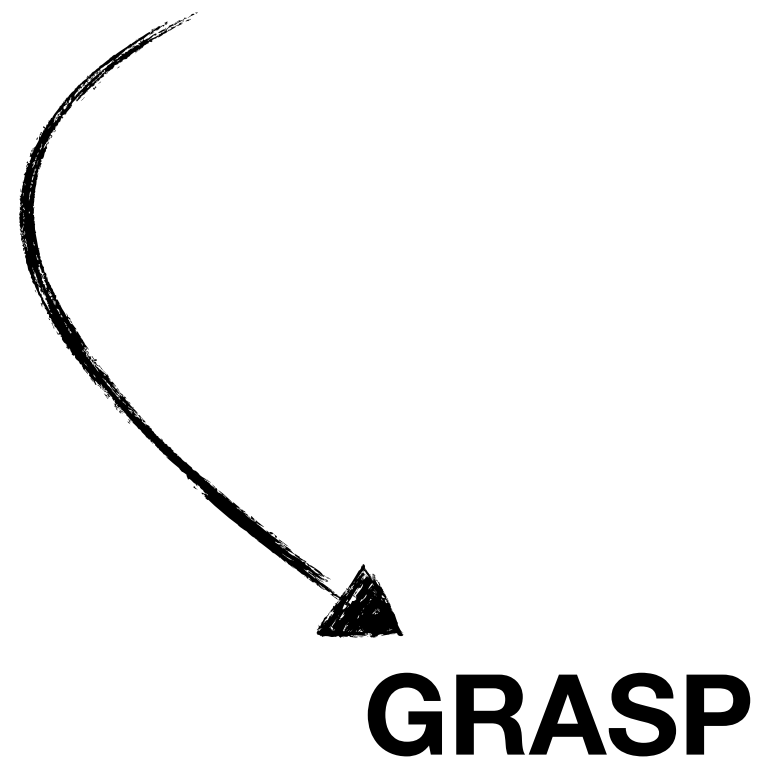


преимущества от соблюдения DIP

- бОльшая гибкость при расширении
- соблюдение OCP

GRASP

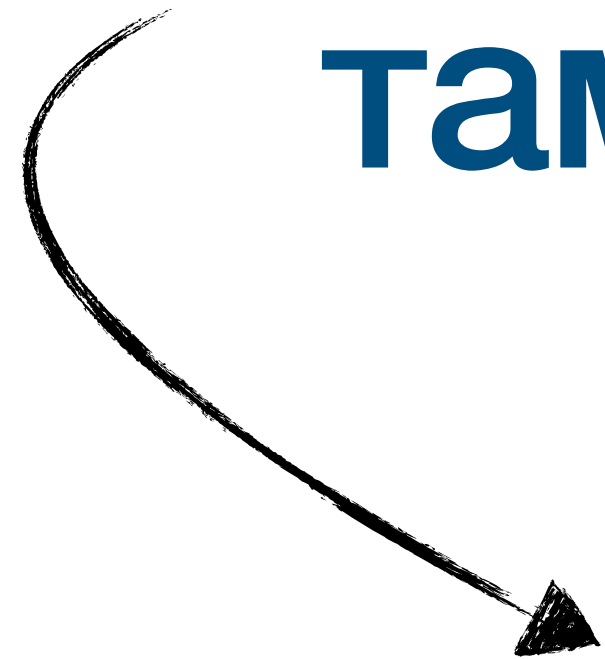
general responsibility assignment software principles
общие принципы распределения ответственностей в ПО



- GRASP содержит много принципов
- некоторая часть из них немного логически устарела
- некоторая часть из них довольно очевидна/покрывается другими принципами
- а некоторая часть довольно релевантна, рассматривает те же проблемы, что и SOLID, но с немного другой стороны

GRASP
information expert

**информация должна обрабатываться
там, где она содержится**



information expert

правила для...

information expert

- корректной инкапсуляции логики
- помогают ответить на вопрос “а где это должно делаться?”

пример нарушения information expert

```
public record OrderItem(  
    int Id,  
    decimal Price,  
    int Quantity);  
  
public class Order  
{  
    private readonly List<OrderItem> _items = [];  
  
    public IReadOnlyCollection<OrderItem> Items => _items;  
}
```

```
public record Receipt(  
    decimal TotalCost,  
    DateTime Timestamp);  
  
public class ReceiptService  
{  
    public Receipt CalculateReceipt(Order customer)  
    {  
        var totalCost = customer.Items  
            .Sum(order => order.Price * order.Quantity);  
  
        var timestamp = DateTime.Now;  
  
        return new Receipt(totalCost, timestamp);  
    }  
}
```

проблемы от нарушения information expert

- нарушение SRP
- проблемы с переиспользованием
 - либо копипаста
 - либо нелогичная связь между модулями
- усложнённое тестирование

как определить нарушение

information expert

- “анемичные модели” – есть данные, но никакой логики
- перегрузка логикой
 - god-object/god-type
 - один тип напрямую работает с данными большого количества других ТИПОВ
- дубликация логики

как исправить нарушение

information expert

- “логика должна находиться там, где находится минимально полный набор данных, необходимый для её реализации”
- т.е. инкапсулировать логику в типах, хранящих данные

пример соблюдения information expert

```
public record OrderItem(  
    int Id,  
    decimal Price,  
    int Quantity)  
{  
    public decimal Cost ⇒ Price * Quantity;  
}  
  
public class Order  
{  
    private readonly List<OrderItem> _items = [];  
  
    public IReadOnlyCollection<OrderItem> Items ⇒ _items;  
  
    public decimal TotalCost ⇒ _items.Sum(x ⇒ x.Cost);  
}
```

```
public record Receipt(  
    decimal TotalCost,  
    DateTime Timestamp);  
  
public class ReceiptService  
{  
    public Receipt CalculateReceipt(Order customer)  
    {  
        var totalCost = customer.TotalCost;  
        var timestamp = DateTime.Now;  
  
        return new Receipt(totalCost, timestamp);  
    }  
}
```


преимущества от соблюдения information expert

- Большая модульность логики
 - проще переиспользовать
 - проще изолированно тестировать
- соблюдение SRP

GRASP

low coupling & high cohesion

**мера зависимости модулей друг
друга**



coupling

**мера логической соотнесённости
логики в рамках модуля**



cohesion

low cohesion

^ плохо

```
public class DataProvider(HttpClient httpClient)
{
    public (double Value, DateTime Timestamp) GetTemperatureData()
    {
        var response = httpClient.Get("http://sensor");
        var value = response.Content.ReadFromJson<double>();

        return (value, DateTime.Now);
    }

    public (double Value, DateTime Timestamp) GetUsedMemoryData()
    {
        return (GC.GetTotalMemory(false), DateTime.Now);
    }
}
```


low cohesion

^ плохо

- нарушение SRP

high cohesion

^ хорошо

```
public class TemperatureDataProvider(HttpClient httpClient)
{
    public (double Value, DateTime Timestamp) GetTemperatureData()
    {
        var response = _httpClient.Get("http://sensor");
        var value = response.Content.ReadFromJson<double>();

        return (value, DateTime.Now);
    }
}

public class UsedMemoryDataProvider
{
    public (double Value, DateTime Timestamp) GetUsedMemoryData()
    {
        return (GC.GetTotalMemory(false), DateTime.Now);
    }
}
```

high coupling

^ плохо

```
public class DataMonitor
{
    private readonly TemperatureDataProvider _temperatureProvider;
    private readonly UsedMemoryDataProvider _usedMemoryProvider;

    ...

    public void Monitor(MetricType type, CancellationToken cancellationToken)
    {
        while (cancellationToken.IsCancellationRequested is false)
        {
            var (value, timestamp) = type switch
            {
                MetricType.Temperature => _temperatureProvider.GetTemperatureData(),
                MetricType.UsedMemory => _usedMemoryProvider.GetUsedMemoryData(),
            };

            Console.WriteLine($"{type} = {value}, at {timestamp}");
        }
    }
}
```


high coupling

^ плохо

- нарушение OCP

low coupling

^ хорошо

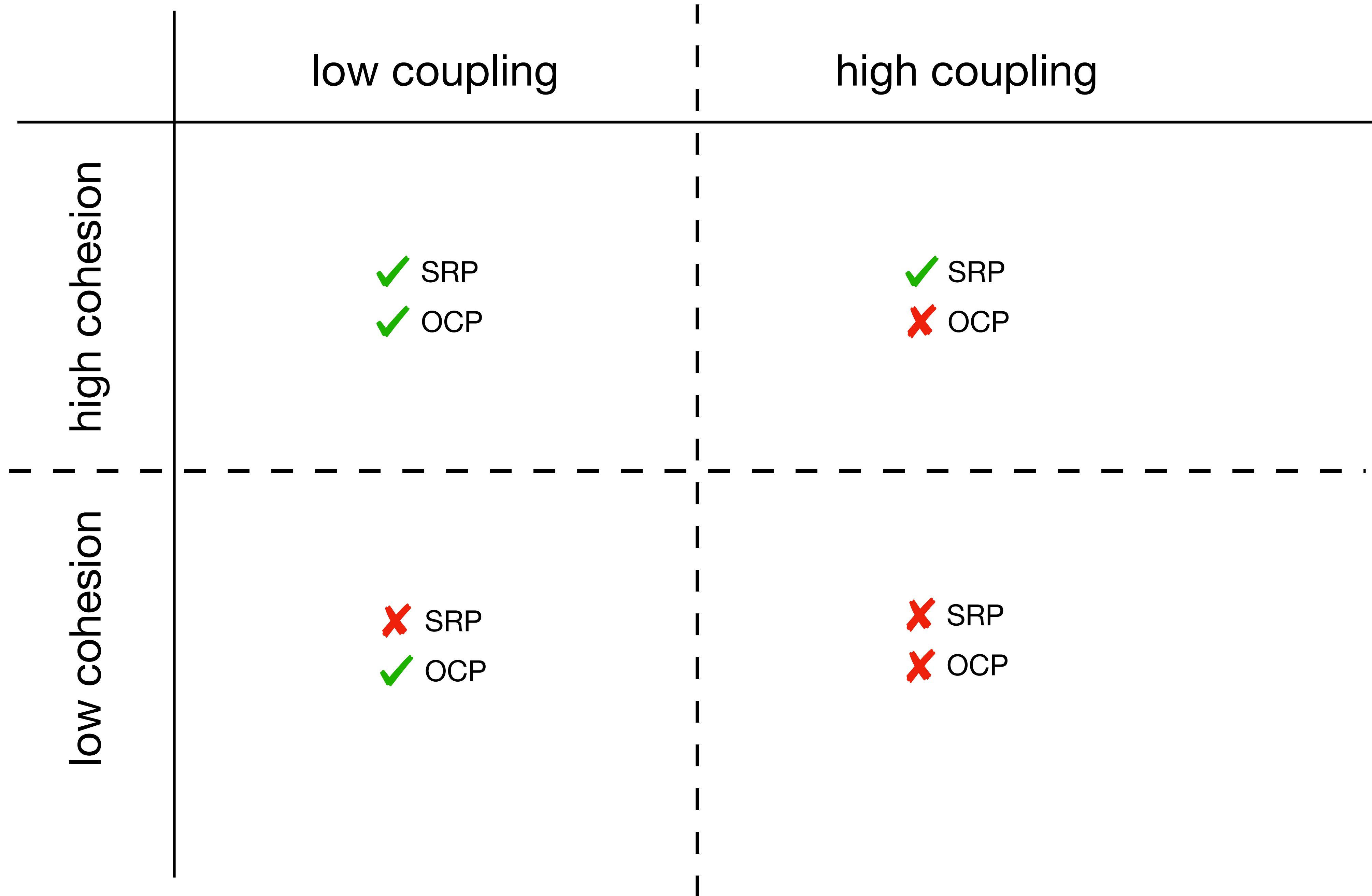
```
public interface IChronologicalDataProvider
{
    string Kind { get; }

    (double Value, DateTime Timestamp) GetData();
}

public class DataMonitor
{
    private readonly IChronologicalDataProvider _provider;

    public DataMonitor(IChronologicalDataProvider provider)
    {
        _provider = provider;
    }

    public void Monitor(CancellationToken cancellationToken)
    {
        while (cancellationToken.IsCancellationRequested is false)
        {
            var (value, timestamp) = _provider.GetData();
            Console.WriteLine($"{_provider.Kind} = {value}, at {timestamp}");
        }
    }
}
```

GRASP
indirection

любое взаимодействие с данными, поведением,
модулями, реализованное не напрямую, а через
какой-либо агрегирующий их объект



object indirection

любое взаимодействие с данными, поведением,
модулями, реализованное не напрямую, а через
какой-либо интерфейс



interface indirection

GRASP

protected variations

protected variations

- коррелирует с OCP из SOLID
делает больший акцент на определение точек нестабильности
- подразумевает выделение стабильного интерфейса над нестабильной реализацией

GRASP

pure fabrication

pure fabrication

- подразумевает наличие в системе искусственной, выдуманной сущности, не отражающей конкретный объект моделируемых бизнес процессов
- обычно это инфраструктуры модули, сервисы
- не рекомендуется вносить такие типы в доменную модель